

Implementation Design

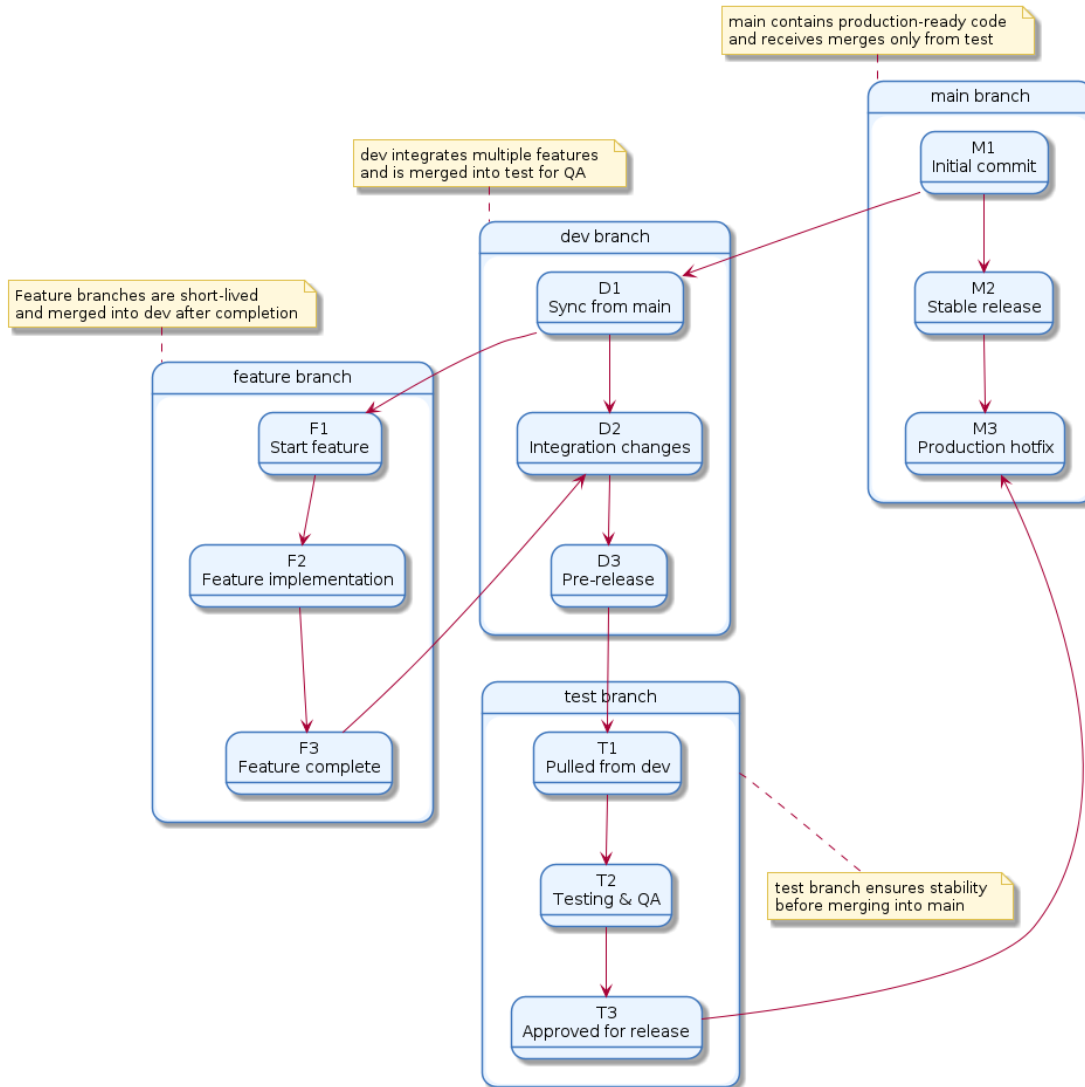
Seth Williams, Abraham Sharum, Parker Fletcher, Bryson Dye

Due Date: October 9, 2025
Class: CS 40003 - Software Engineering

October 9, 2025

GitTea Repository Structure

Git Branching Structure: main, dev, test, and feature



Coding Standards

PHP Standards

- Opening braces are on the same line as declarations
- One class per file, file name matches class name
- Naming conventions:
 - Classes: `PascalCase`
 - Methods: `camelCase`
 - Variables: `$camelCase`
 - Constants: `UPPER_SNAKE_CASE`

HTML Standards

- Use lowercase for all element and attribute names
- Indent nested elements by 2 spaces
- Use double quotes for all attribute values
- Use semantic tags for structure: `<header>`, `<main>`, `<footer>`, `<section>`.
- Always include alt text for images
- Close all elements, including void elements like `<br>` and `<img>`
- Separate structure (HTML), style (CSS), and logic (JavaScript)

JavaScript Standards

- Indentation: 2 spaces
- Use `const` and `let` instead of `var`
- Use single quotes for strings
- Always use semicolons
- Use strict equality (`===` and `!==`)
- Avoid global variables
- Naming conventions:
 - Variables and functions: `camelCase`
 - Classes and constructors: `PascalCase`
 - Constants: `UPPER_SNAKE_CASE`
- Use meaningful variable names, avoid single letter identifiers outside of indices

SQL Standards

- Use uppercase for all SQL keywords
- Use lowercase with underscores for table and column names
- Use consistent indentation and line breaks for readability:
 - Each major clause (`SELECT`, `FROM`, `WHERE`, `GROUP BY`, etc.) on a new line
 - Align columns in `SELECT` statements vertically
- Always end statements with a semicolon
- Avoid using `SELECT *`, explicitly list required columns
- Use aliases with the `AS` keyword for clarity
- Use foreign key constraints for relational integrity

Security Standards

- Validate all user input on both client and server sides
- Use prepared statements to prevent SQL injection
- Hash all stored passwords
- Use HTTPS for secure communication between clients and servers
- Store API keys and credentials in environment variables, not in the repository
- Limit user permissions and apply role-based access control
- Implement secure session handling with timeouts and token regeneration
- Do not expose detailed error messages to users